AetherHound — build manual

Step by step, in order. Every step has a command and a verify.

> **Rules for using this.** Steps are numbered and ordered. Do not skip forward — later steps import earlier files. Each step ends with verify: if it fails, fix it before continuing. Steps marked cut can be dropped if the clock beats you; nothing else can.

Contents

Phase 0 — machine setup (steps 0.1–0.9) 2

Phase 1 — the transmitter (1.1–1.8) 4

Phase 2 — the recorder (2.1–2.6) 7

Phase 3 — baseline capture (3.1–3.5) 9

Phase 4 — signal conditioning (4.1–4.10) 10

Phase 5 — dataset assembly (5.1–5.8) 13

Phase 6 — autoencoder (6.1–6.10) 15

Phase 7 — isolation forest baseline (7.1–7.4) 18

Phase 8 — live detection engine (8.1–8.9) 19

Phase 9 — terminal UI (9.1–9.6) 23

Phase 10 — Rust port (10.1–10.9) 25

Phase 11 — deployment (11.1–11.6) 28

Phase 12 — evaluation (12.1–12.7) 29

Phase 13 — freeze and hand over (13.1–13.4) 32

Compressed 5-day map 33

Phase 0 — machine setup

Target: 40 minutes. **Output:** a repo that imports cleanly and a mic that records.

0.1 Choose the machine

Linux, native or WSL2 with audio passthrough. Windows-native works for Phases 1–8 but device handling is worse. Note the mic device index you will use.

0.2 Install system packages

```
sudo apt update
sudo apt install -y git python3-venv python3-pip ffmpeg sox \
  libportaudio2 portaudio19-dev sqlite3 build-essential \
  librtlsdr-dev rtl-sdr gnuradio gqrx-sdr
```

Verify: `sox --version` prints. `rtl_test -t` prints either a device or "no supported devices" — either is fine, you are not blocked on RF.

0.3 Make the repo

```
mkdir -p ~/aetherhound && cd ~/aetherhound
git init
mkdir -p mock capture dsp train export infer ui eval deploy models \
  data/raw data/baseline data/test docs tests
touch README.md LICENSE
```

0.4 .gitignore

```
cat > .gitignore <<'EOF'
.venv/
__pycache__/
*.pyc
data/
models/*.pt
models/*.onnx
*.db
*.db-wal
*.db-shm
docs/*.pdf
EOF
```

0.5 Python environment

```
python3 -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install numpy scipy soundfile sounddevice librosa matplotlib \
  scikit-learn torch onnx onnxruntime rich pytest
pip freeze > requirements.txt
```

Verify: `python -c "import torch, librosa, sounddevice; print(torch.__version__)"`

0.6 Config file

```
cat > config.toml <<'EOF'
[audio]
device = -1          # -1 = system default; set an index from 0.7
rate = 48000
blocksize = 1024
channels = 1

[band]
lo_hz = 300
hi_hz = 8000

[dsp]
notch_hz = [50, 100, 150, 200]
nfft = 512
hop = 32
patch = 64

[detect]
threshold_file = "models/threshold.txt"
armed_frames = 2
clear_frames = 5
hold_s = 1.5
clear_ratio = 0.7

[log]
db = "events.db"

[rf]
enabled = false
center_hz = 100000000
gain = 28
EOF
```

0.7 Find your mic index

```
python - <<'EOF'
import sounddevice as sd
for i, d in enumerate(sd.query_devices()):
    if d['max_input_channels'] > 0:
        print(i, d['name'], d['default_samplerate'])
EOF
```

Put that index in `config.toml` under device.

> **Verify:** the index prints and the name is your built-in or headset mic. If sample rate maxes at 44100, that is fine — set `rate = 44100`.

0.8 Input level

```
python - <<'EOF'
import sounddevice as sd, numpy as np, time
def cb(d, f, t, s):
    print(f"peak {np.abs(d).max():.4f}  rms {np.sqrt((d**2).mean()):.5f}", end="\r")
with sd.InputStream(channels=1, callback=cb):
    time.sleep(8)
EOF
```

Quiet room should sit near rms 0.001–0.01. If peak hits 1.0000 you are clipping — lower input gain in the OS mixer. Too quiet and the fan signal disappears.

0.9 First commit

```
git add -A && git commit -m "scaffold: dirs, config, env"
```

Phase 1 — the transmitter

Target: 2 hours. **Output:** a script that makes the machine emit Morse.

1.1 Check for fan PWM

```
ls /sys/class/hwmon/hwmon*/pwm1 2>/dev/null
for f in /sys/class/hwmon/hwmon*/pwm1; do echo "$f -> $(cat $(dirname $f)/name)"; done
```

If any path prints, PWM mode works and you are lucky. If nothing prints, the CPU-load fallback is your transmitter — that is fine and expected on most laptops.

1.2 Write the transmitter

```
cat > mock/fan_morse.py <<'PYEOF'
#!/usr/bin/env python3
"""Mock Fansmitter. PWM if available, CPU-load ramp otherwise."""
import argparse, glob, json, multiprocessing as mp, os, sys, time

DOT, DASH, GAP, LETTER, WORD = 0.20, 0.60, 0.20, 0.60, 1.40
MORSE = {'A':'.-','B':'-...','C':'-.-.','D':'-..','E':'.','F':'..-.','G':'--.',
         'H':'....','I':'..','J':'.---','K':'-.-','L':'.-..','M':'--','N':'-.',
         'O':'---','P':'.--.','Q':'--.-','R':'.-.','S':'...','T':'-','U':'..-',
         'V':'...-','W':'.--','X':'-..-','Y':'-.--','Z':'--..',
         '0':'-----','1':'.----','2':'..---','3':'...--','4':'....-',
         '5':'.....','6':'-....','7':'--...','8':'---..','9':'----.'}

def find_pwm():
```

```
    for p in sorted(glob.glob('/sys/class/hwmon/hwmon*/pwm1')):
        if os.path.exists(os.path.join(os.path.dirname(p), 'pwm1_enable')):
            return p
    return None

def write(p, v):
    try:
        open(p, 'w').write(str(v))
    except OSError:
        pass

def _burn(stop):
    while not stop.is_set():
        pass

class LoadTx:
    def __init__(self, n): self.n, self.p = n, []
    def on(self):
        self.p = [mp.Process(target=_burn, args=(mp.Event(),)) for _ in range(self.n)]
        for q in self.p: q.start()
    def off(self):
        for q in self.p: q.terminate()
        self.p = []

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('text', nargs='?', default='SOS')
    ap.add_argument('--pwm', action='store_true')
    ap.add_argument('--cores', type=int, default=os.cpu_count() or 2)
    ap.add_argument('--lo', type=int, default=120)
    ap.add_argument('--hi', type=int, default=255)
    ap.add_argument('--gt', default='data/groundtruth.jsonl')
    ap.add_argument('--gap', type=float, default=0.0,
                    help='seconds of silence between letters')
    a = ap.parse_args()

    pwm = find_pwm() if a.pwm else None
    mode = 'pwm' if pwm else 'load'
    tx = None
    if mode == 'pwm':
        write(os.path.join(os.path.dirname(pwm), 'pwm1_enable'), 1)
    else:
        if a.pwm:
            print('no pwm1 -> CPU load fallback', file=sys.stderr)
        tx = LoadTx(a.cores)

    os.makedirs(os.path.dirname(a.gt) or '.', exist_ok=True)
    gt = open(a.gt, 'a')

    def key(down):
        if mode == 'pwm':
            write(pwm, a.hi if down else a.lo)
        elif down:
            tx.on()
        else:
            tx.off()

    gt.write(json.dumps({'t': time.time(), 'event': 'start',
                         'mode': mode, 'text': a.text}) + '\n')
    print(f'mode={mode} text={a.text!r}')
    try:
        for w in a.text.upper().split():
            for c in w:
```

```
                for sym in MORSE.get(c, ''):
                    key(True);  time.sleep(DOT if sym == '.' else DASH)
                    key(False); time.sleep(GAP)
                time.sleep(LETTER)
            time.sleep(WORD)
    except KeyboardInterrupt:
        pass
    finally:
        key(False)
        gt.write(json.dumps({'t': time.time(), 'event': 'end', 'mode': mode}) + '\n')
        gt.close()
        print('\ndone')

if __name__ == '__main__':
    main()
PYEOF
chmod +x mock/fan_morse.py
```

1.3 Test it

```
python mock/fan_morse.py --pwm "SOS"
```

Verify: you hear the fan pitch change, or the CPU spins up audibly. If neither, run without `--pwm` and with `--cores 4`.

1.4 Tune amplitude

If the fan barely moves, raise `--cores` to all cores. If the fan is always spinning hard anyway, widen the duty range: `--lo 60 --hi 255`.

1.5 Check the ground-truth log

```
cat data/groundtruth.jsonl
```

Verify: one start line and one end line, with end > start.

1.6 Write a random-interval driver

```
cat > mock/drive.py <<'PYEOF'
#!/usr/bin/env python3
"""Fire the transmitter at random intervals for a fixed duration."""
import argparse, random, subprocess, sys, time

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--dur', type=float, default=3600)
    ap.add_argument('--lo', type=float, default=20)
    ap.add_argument('--hi', type=float, default=90)
    ap.add_argument('--words', default='SOS KEY DATA')
    a = ap.parse_args()
    end = time.time() + a.dur
```

```
    while time.time() < end:
        time.sleep(random.uniform(a.lo, a.hi))
        if time.time() >= end: break
        subprocess.run([sys.executable, 'mock/fan_morse.py', a.words])

if __name__ == '__main__':
    main()
PYEOF
```

1.7 Test the driver for 2 minutes

```
python mock/drive.py --dur 120
```

1.8 Commit

```
git add mock/ && git commit -m "mock: fansmitter pwm + load fallback, random driver"
```

Phase 2 — the recorder

Target: 45 minutes. **Output:** clean timestamped WAVs.

2.1 Write the recorder

```
cat > capture/rec.py <<'PYEOF'
#!/usr/bin/env python3
import argparse, datetime, json, os, sounddevice as sd, soundfile as sf

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--dur', type=float, default=60)
    ap.add_argument('--rate', type=int, default=48000)
    ap.add_argument('--dev', type=int, default=None)
    ap.add_argument('--out', default='data/raw')
    a = ap.parse_args()
    os.makedirs(a.out, exist_ok=True)
    ts = datetime.datetime.now().strftime('%Y%m%d-%H%M%S')
    path = os.path.join(a.out, ts + '.wav')
    drops = 0

    with sf.SoundFile(path, 'w', a.rate, 1, 'FLOAT') as f:
        def cb(indata, frames, t, status):
            nonlocal drops
            if status:
                drops += 1
                print(status, file=os.sys.stderr)
            f.write(indata)
        with sd.InputStream(device=a.dev, channels=1, samplerate=a.rate,
                            dtype='float32', callback=cb, blocksize=1024):
            sd.sleep(int(a.dur * 1000))

    meta = {'path': path, 'start': ts, 'rate': a.rate, 'dev': a.dev,
            'drops': drops, 'dur': a.dur}
```

```
    open(path + '.json', 'w').write(json.dumps(meta))
    print(path, 'drops=', drops)

if __name__ == '__main__':
    main()
PYEOF
```

2.2 Record 10 seconds

```
python capture/rec.py --dur 10 --out data/raw
```

Verify: a path prints and drops=0. Play it back: `sox data/raw/*.wav -d`.

2.3 Inspect levels

```
sox data/raw/*.wav -n stat 2>&1 | grep -E "RMS|Maximum"
```

Maximum should be well under 1.0 and RMS above 0.0005. Adjust if not.

2.4 Record 10 s with the transmitter running

```
python capture/rec.py --dur 10 &
python mock/fan_morse.py --pwm "SOS"
wait
```

Note which file that is; you will plot it in 4.9.

2.5 Drop counter

If drops > 0 in any file, raise blocksize to 2048 in the recorder call, or close other apps. A file with drops over 5 is unusable — delete it.

2.6 Commit

```
git add capture/ && git commit -m "capture: timestamped wav recorder with drop counter"
```

Phase 3 — baseline capture

Target: 2 hours of wall clock, mostly unattended. **Output:** 120 minutes of clean negatives.

3.1 Baseline driver

```
cat > capture/baseline.sh <<'EOF'
#!/usr/bin/env bash
set -u
mkdir -p data/baseline
for i in $(seq 1 20); do
  echo "chunk $i at $(date -Is)" | tee -a data/baseline/session.log
  python capture/rec.py --dur 360 --out data/baseline || echo "fail $i" >> data/baseline/session.log
  sleep 2
done
EOF
chmod +x capture/baseline.sh
```

3.2 Conditions

Room empty. Your machine idle. No music, no fan pointed at the mic. Write ambient temperature in the session log at start and end. If the machine's own fan is running, note it — that is your baseline, not noise to eliminate.

3.3 Run it

```
./capture/baseline.sh
```

Walk away. Do not touch the laptop. A fan cycle mid-capture is fine and realistic.

3.4 Verify coverage

```
ls data/baseline/*.wav | wc -l
soxi -T data/baseline/*.wav | paste -sd+ | bc
```

Verify: 20 files, total duration ≥ 7000 seconds. Delete any file with drops > 5 in its `.json`.

3.5 Commit the log only

```
git add capture/baseline.sh && git commit -m "capture: baseline driver"
```

Phase 4 — signal conditioning

Target: 4 hours. **Output:** normalised spectrogram patches and a spectral baseline.

4.1 Filters

```
cat > dsp/filters.py <<'PYEOF'
import numpy as np
from scipy.signal import iirnotch, butter, sosfiltfilt, tf2sos

def notch(x, freqs, sr, q=30.0):
    for f in freqs:
        if f < sr / 2 - 50:
            b, a = iirnotch(f, q, sr)
            x = sosfiltfilt(tf2sos(b, a), x)
    return x

def bandpass(x, lo, hi, sr, order=4):
    lo = max(lo, 20.0)
    hi = min(hi, sr / 2 - 200)
    sos = butter(order, [lo / (sr / 2), hi / (sr / 2)], btype='band', output='sos')
    return sosfiltfilt(sos, x)

def dehum(x, sr, freqs):
    return notch(x, freqs, sr)
PYEOF
```

4.2 Front end

```
cat > dsp/frontend.py <<'PYEOF'
import numpy as np, librosa
from dsp.filters import bandpass, dehum

def prep(x, sr, cfg):
    x = np.asarray(x, dtype='float32')
    x = x - x.mean()
    x = dehum(x, sr, cfg['dsp']['notch_hz'])
    x = bandpass(x, cfg['band']['lo_hz'], cfg['band']['hi_hz'], sr)
    return x

def spec(x, cfg):
    nfft, hop = cfg['dsp']['nfft'], cfg['dsp']['hop']
    S = np.abs(librosa.stft(x, n_fft=nfft, hop_length=hop,
                            win_length=nfft, window='hann'))
    return librosa.amplitude_to_db(S + 1e-10, ref=np.max)

def norm(S):
    return ((S - S.mean()) / (S.std() + 1e-9)).astype('float32')

def patches(S, size):
    out = []
    for i in range(0, S.shape[1] - size, size // 2):
        out.append(S[:size, i:i + size])
    return np.stack(out) if out else np.empty((0, size, size), 'float32')
PYEOF
```

4.3 Spectral baseline

```
cat > dsp/spectral_sub.py <<'PYEOF'
import glob, json, numpy as np, soundfile as sf
from dsp.frontend import prep, spec

def build(cfg, out='models/baseline_spec.npy', limit=30):
    acc, n = None, 0
    for p in sorted(glob.glob('data/baseline/*.wav'))[:limit]:
        x, sr = sf.read(p, dtype='float32')
        S = spec(prep(x, sr, cfg), cfg)
        acc = S if acc is None else acc + S
        n += 1
    if acc is None:
        raise SystemExit('no baseline files')
    m = (acc / n).astype('float32')
    np.save(out, m)
    print('baseline shape', m.shape, '->', out)

if __name__ == '__main__':
    import tomllib
    cfg = tomllib.load(open('config.toml', 'rb'))
    build(cfg)
PYEOF
python dsp/spectral_sub.py
```

Verify: prints a shape matching your `nfft/2+1` by frames, and writes the file.

4.4 Subtract and normalise

```
cat >> dsp/frontend.py <<'PYEOF'

def residual(S, base):
    n = min(S.shape[0], base.shape[0])
    return np.maximum(S[:n] - base[:n], 0.0)
PYEOF
```

4.5 Batch convert

```
cat > dsp/batch.py <<'PYEOF'
import glob, os, sys, numpy as np, soundfile as sf, tomllib
from dsp.frontend import prep, spec, norm, patches, residual

cfg = tomllib.load(open('config.toml', 'rb'))
base = np.load('models/baseline_spec.npy')
size = cfg['dsp']['patch']

def one(path):
    x, sr = sf.read(path, dtype='float32')
    S = spec(prep(x, sr, cfg), cfg)
    P = patches(norm(residual(S, base)), size)
    out = path.replace('.wav', '.npy')
    np.save(out, P)
    return out, P.shape

if __name__ == '__main__':
    for d in sys.argv[1:]:
```

```
        for p in sorted(glob.glob(os.path.join(d, '*.wav'))):
            o, s = one(p)
            print(o, s)
PYEOF
python dsp/batch.py data/raw
```

4.6 Sanity-check the values

```
python - <<'EOF'
import glob, numpy as np
for p in glob.glob('data/raw/*.npy'):
    a = np.load(p)
    print(p, a.shape, round(float(a.mean()), 3), round(float(a.std()), 3),
          round(float(a.min()), 2), round(float(a.max()), 2))
EOF
```

Mean near 0, std near 1. If std is 0.0 the file is silent — drop it.

4.7 Convert the baseline too

```
python dsp/batch.py data/baseline
```

4.8 Record and convert positives

```
mkdir -p data/pos
python capture/rec.py --dur 300 --out data/pos &
python mock/drive.py --dur 300 --lo 15 --hi 40
wait
python dsp/batch.py data/pos
```

4.9 The proof plot

```
cat > eval/plot_proof.py <<'PYEOF'
import glob, sys, numpy as np, matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt, soundfile as sf, tomllib
from dsp.frontend import prep, spec

cfg = tomllib.load(open('config.toml', 'rb'))
neg = sorted(glob.glob('data/baseline/*.wav'))[0]
pos = sorted(glob.glob('data/pos/*.wav'))[0]
fig, ax = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
for a, p, t in zip(ax, (neg, pos), ('baseline', 'transmitting')):
    x, sr = sf.read(p, dtype='float32')
    a.imshow(spec(prep(x[:sr * 8], sr, cfg), cfg), origin='lower',
             aspect='auto', cmap='magma')
    a.set_title(t)
    a.set_xlabel('frame')
ax[0].set_ylabel('bin')
plt.tight_layout()
plt.savefig('docs/proof.png', dpi=140)
print('docs/proof.png')
PYEOF
python eval/plot_proof.py
```

> **Verify:** open the PNG. The right panel must show structure the left does not — a bright horizontal band, or blinking vertical stripes. If both look identical, your bandpass is wrong or the fan is too quiet. Fix before Phase 5.

4.10 Commit

```
git add dsp/ eval/ && git commit -m "dsp: filters, stft frontend, spectral subtraction, proof plot"
```

Phase 5 — dataset assembly

Target: 3 hours. **Output:** `train.npz` and `test.npz` with time-based splits.

5.1 Label from ground truth

```
cat > train/label.py <<'PYEOF'
import datetime, json, os

def epoch_from_name(p):
    b = os.path.basename(p)[:15]
    return datetime.datetime.strptime(b, '%Y%m%d-%H%M%S').timestamp()

def spans(path='data/groundtruth.jsonl', pad=0.5):
    out, start = [], None
    if not os.path.exists(path):
        return out
    for line in open(path):
        d = json.loads(line)
        if d['event'] == 'start':
            start = d['t']
        elif d['event'] == 'end' and start:
            out.append((start - pad, d['t'] + pad))
            start = None
    return out

def hit(t, sp):
    return int(any(a <= t <= b for a, b in sp))
PYEOF
```

5.2 Build the arrays

```
cat > train/dataset.py <<'PYEOF'
import glob, json, os, numpy as np
from train.label import epoch_from_name, spans, hit

HOP_S = 32 * 32 / 48000.0          # 0.02133 s per patch

def collect(pat, sp):
    X, y = [], []
    for p in sorted(glob.glob(pat)):
        a = np.load(p)
        if a.size == 0:
            continue
```

```
        t0 = epoch_from_name(p)
        for i, pt in enumerate(a):
            X.append(pt)
            y.append(hit(t0 + i * HOP_S, sp))
    if not X:
        raise SystemExit('no patches in ' + pat)
    return np.stack(X)[..., None].astype('float32'), np.array(y, 'int64')

if __name__ == '__main__':
    sp = spans()
    print('spans', len(sp))
    Xb, yb = collect('data/baseline/*.npy', sp)
    Xp, yp = collect('data/pos/*.npy', sp)
    X = np.concatenate([Xb, Xp]); y = np.concatenate([yb, yp])
    order = np.argsort(np.concatenate([
        [epoch_from_name(p) for p in sorted(glob.glob('data/baseline/*.npy'))]
        for _ in [0]] * 1 + [[]])[:len(Xb)] if False else np.arange(len(X)))
    cut = int(len(X) * 0.7)
    np.savez_compressed('data/train.npz', X=X[:cut], y=y[:cut])
    np.savez_compressed('data/test.npz', X=X[cut:], y=y[cut:])
    print('train', X[:cut].shape, int(y[:cut].sum()),
          'test', X[cut:].shape, int(y[cut:].sum()))
PYEOF
python train/dataset.py
```

5.3 Verify balance

Verify: the printed positive counts are not zero in either split. A test set with zero positives means you recorded too few positive files — go back to 4.8 and record three more minutes.

5.4 Augmentation

```
cat > train/augment.py <<'PYEOF'
import numpy as np

def aug(P, rng):
    P = P.copy()
    if rng.random() < 0.5:
        P *= rng.uniform(0.5, 1.6)                    # gain
    if rng.random() < 0.5:
        P = np.roll(P, rng.integers(-8, 8), axis=1)    # time shift
    if rng.random() < 0.4:
        P += rng.normal(0, rng.uniform(0.05, 0.3), P.shape).astype('float32')
    return P

def expand(X, y, factor, seed=0):
    rng = np.random.default_rng(seed)
    out = [X]
    for _ in range(factor - 1):
        out.append(np.stack([aug(p[..., 0], rng)[..., None] for p in X]))
    return np.concatenate(out), np.tile(y, factor)
PYEOF
```

5.5 Augment the training set only

```
python - <<'EOF'
import numpy as np
from train.augment import expand
d = np.load('data/train.npz')
X, y = expand(d['X'], d['y'], 3)
np.savez_compressed('data/train_aug.npz', X=X, y=y)
print(X.shape, int(y.sum()))
EOF
```

5.6 Never augment the test set

The test set is the held-out session. Augmenting it flatters the numbers.

5.7 Commit

```
git add train/ && git commit -m "train: dataset assembly, labels, augmentation"
```

5.8 Statistics for the report

```
python - <<'EOF'
import numpy as np
for n in ('train_aug', 'test'):
    d = np.load(f'data/{n}.npz')
    print(n, d['X'].shape, 'pos', int(d['y'].sum()), 'ratio',
          round(float(d['y'].sum()) / len(d['y']), 4))
EOF
```

Copy the output into `docs/log.md`. These are report numbers.

Phase 6 — autoencoder

Target: 4 hours. **Output:** trained model, calibrated threshold, ONNX file.

6.1 Model

```
cat > train/model.py <<'PYEOF'
import torch, torch.nn as nn

class AE(nn.Module):
    def __init__(self, size=64):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(1, 16, 3, 2, 1), nn.BatchNorm2d(16), nn.LeakyReLU(0.2),
            nn.Conv2d(16, 32, 3, 2, 1), nn.BatchNorm2d(32), nn.LeakyReLU(0.2),
            nn.Conv2d(32, 64, 3, 2, 1), nn.BatchNorm2d(64), nn.LeakyReLU(0.2))
        self.dec = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 3, 2, 1, 1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.ConvTranspose2d(32, 16, 3, 2, 1, 1), nn.BatchNorm2d(16), nn.ReLU(),
```

```
            nn.ConvTranspose2d(16, 1, 3, 2, 1, 1), nn.Sigmoid())
    def forward(self, x):
        return self.dec(self.enc(x))

def errs(model, X, dev, bs=256):
    model.eval()
    out = []
    with torch.no_grad():
        for i in range(0, len(X), bs):
            b = torch.tensor(X[i:i + bs], dtype=torch.float32).to(dev)
            out.append(((model(b) - b) ** 2).mean((1, 2, 3)).cpu().numpy())
    return np.concatenate(out) if out else np.array([])

import numpy as np
PYEOF
```

6.2 Trainer

```
cat > train/fit.py <<'PYEOF'
import argparse, numpy as np, torch, torch.nn as nn
from train.model import AE, errs

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--epochs', type=int, default=40)
    ap.add_argument('--bs', type=int, default=128)
    ap.add_argument('--lr', type=float, default=1e-3)
    ap.add_argument('--out', default='models/ae.pt')
    a = ap.parse_args()

    d = np.load('data/train_aug.npz')
    neg = d['X'][d['y'] == 0]
    rng = np.random.default_rng(0); rng.shuffle(neg)
    val = neg[:len(neg) // 10]; neg = neg[len(neg) // 10:]
    print('neg', neg.shape, 'val', val.shape)

    dev = 'cuda' if torch.cuda.is_available() else 'cpu'
    m = AE().to(dev)
    opt = torch.optim.Adam(m.parameters(), a.lr)
    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, patience=3, factor=0.5)
    lossf = nn.MSELoss()
    X = torch.tensor(neg, dtype=torch.float32)
    best, bad = 1e9, 0

    for e in range(a.epochs):
        m.train(); perm = torch.randperm(len(X)); tot = 0.0
        for i in range(0, len(X), a.bs):
            b = X[perm[i:i + a.bs]].to(dev)
            opt.zero_grad(); l = lossf(m(b), b); l.backward(); opt.step()
            tot += l.item() * len(b)
        v = float(errs(m, val, dev).mean())
        sched.step(v)
        print(f'epoch {e} train {tot / len(X):.6f} val {v:.6f}')
        if v < best - 1e-6:
            best, bad = v, 0
            torch.save(m.state_dict(), a.out)
        else:
            bad += 1
            if bad >= 8:
                print('early stop'); break
```

```
    print('best', best, '->', a.out)

if __name__ == '__main__':
    main()
PYEOF
```

6.3 Train

```
python -m train.fit --epochs 40
```

Verify: val loss descends then flattens. A flat line from epoch 0 means the patches carry no structure — return to 4.9.

6.4 Calibrate the threshold

```
cat > train/calibrate.py <<'PYEOF'
import numpy as np, torch, tomllib
from train.model import AE, errs

cfg = tomllib.load(open('config.toml', 'rb'))
dev = 'cpu'
m = AE(); m.load_state_dict(torch.load('models/ae.pt', map_location='cpu')); m.eval()
d = np.load('data/train.npz')
neg = d['X'][d['y'] == 0]
e = errs(m, neg, dev)
pct = 99.5
thr = float(np.percentile(e, pct))
open('models/threshold.txt', 'w').write(f'{thr:.8f}')
np.save('models/err_dist.npy', e)
print('n', len(e), 'p50', float(np.percentile(e, 50)), 'thr', thr)
PYEOF
python train/calibrate.py
```

Verify: threshold is clearly above the median error. If it is close to the median, the model is underfit.

6.5 Export ONNX

```
cat > export/to_onnx.py <<'PYEOF'
import torch, onnx
from train.model import AE
m = AE(); m.load_state_dict(torch.load('models/ae.pt', map_location='cpu')); m.eval()
torch.onnx.export(m, torch.randn(1, 1, 64, 64), 'models/ae.onnx',
                  input_names=['x'], output_names=['y'],
                  dynamic_axes={'x': {0: 'n'}, 'y': {0: 'n'}}, opset_version=17)
onnx.checker.check_model('models/ae.onnx')
print('ok')
PYEOF
python export/to_onnx.py
```

6.6 Parity check

```
python - <<'EOF'
import numpy as np, onnxruntime as ort, torch
from train.model import AE
d = np.load('data/test.npz'); b = d['X'][:16]
m = AE(); m.load_state_dict(torch.load('models/ae.pt', map_location='cpu')); m.eval()
with torch.no_grad():
    a = ((m(torch.tensor(b)) - torch.tensor(b)) ** 2).mean((1, 2, 3)).numpy()
s = ort.InferenceSession('models/ae.onnx', providers=['CPUExecutionProvider'])
o = s.run(None, {'x': b})[0]
c = ((o - b) ** 2).mean((1, 2, 3))
print('max abs diff', float(np.abs(a - c).max()))
EOF
```

Verify: diff under 1e-4. Above that, the export lost precision — re-export with opset 13.

6.7 Score the test set

```
python - <<'EOF'
import numpy as np, onnxruntime as ort
d = np.load('data/test.npz')
s = ort.InferenceSession('models/ae.onnx', providers=['CPUExecutionProvider'])
o = s.run(None, {'x': d['X']})[0]
sc = ((o - d['X']) ** 2).mean((1, 2, 3))
np.save('models/test_scores.npy', sc)
print('pos mean', float(sc[d['y'] == 1].mean()), 'neg mean', float(sc[d['y'] == 0].mean()))
EOF
```

Verify: positive mean is clearly above negative mean. If they overlap, you have a real problem — check the proof plot again.

6.8 Commit

```
git add train/ export/ && git commit -m "model: ae, training, threshold calibration, onnx export"
```

Phase 7 — isolation forest baseline

Target: 40 minutes. **Output:** a comparison row.

7.1 Features

```
cat > train/forest.py <<'PYEOF'
import numpy as np
from sklearn.ensemble import IsolationForest

def feats(X):
    X = X[..., 0]
    return np.column_stack([
```

```
        X.mean((1, 2)), X.std((1, 2)), np.percentile(X, 90, (1, 2)),
        np.abs(np.diff(X, axis=2)).mean((1, 2)),
        np.abs(np.diff(X, axis=1)).mean((1, 2)),
        np.percentile(X, 99, (1, 2)),
    ])

def fit_score(train, test):
    tr = np.load(train); te = np.load(test)
    m = IsolationForest(contamination=0.02, n_estimators=300, random_state=0)
    m.fit(feats(tr['X'][tr['y'] == 0]))
    return -m.score_samples(feats(te['X'])), te['y']
PYEOF
python - <<'EOF'
import numpy as np
from train.forest import fit_score
s, y = fit_score('data/train_aug.npz', 'data/test.npz')
np.save('models/forest_scores.npy', s)
print('pos mean', float(s[y == 1].mean()), 'neg mean', float(s[y == 0].mean()))
EOF
```

7.2 Expect the forest to lose

It usually catches the steady tone and misses the ramps. That is the point of the comparison.

7.3 Commit

```
git add train/forest.py && git commit -m "train: isolation forest baseline"
```

Phase 8 — live detection engine

Target: 5 hours. **Output:** a process that alarms in under two seconds.

8.1 Event log

```
cat > infer/db.py <<'PYEOF'
import sqlite3, time

DDL = """CREATE TABLE IF NOT EXISTS events(
  id INTEGER PRIMARY KEY, t REAL NOT NULL, state TEXT NOT NULL,
  score REAL NOT NULL, thr REAL NOT NULL, note TEXT)"""

def open_db(p):
    c = sqlite3.connect(p, isolation_level=None)
    c.execute('PRAGMA journal_mode=WAL')
    c.execute(DDL)
    return c

def add(c, state, score, thr, note=''):
    c.execute('INSERT INTO events(t,state,score,thr,note) VALUES(?,?,?,?,?)',
              (time.time(), state, score, thr, note))
PYEOF
```

8.2 State machine

```
cat > infer/state.py <<'PYEOF'
import time

IDLE, ARMED, ALARM = 'idle', 'armed', 'alarm'

class Detector:
    """Hysteresis + debounce. Two over-threshold frames arm; five
    consecutive quiet frames clear, and only after the hold time."""
    def __init__(self, thr, armed_n=2, clear_n=5, hold=1.5, ratio=0.7):
        self.thr, self.armed_n, self.clear_n = thr, armed_n, clear_n
        self.hold, self.clear_at = hold, thr * ratio
        self.state = IDLE
        self.over = self.under = 0
        self.t_alarm = 0.0

    def step(self, score, now=None):
        now = now or time.time()
        fired = False
        if score >= self.thr:
            self.over += 1; self.under = 0
        elif score < self.clear_at:
            self.under += 1; self.over = 0

        if self.state == IDLE and self.over >= self.armed_n:
            self.state = ARMED
        elif self.state == ARMED:
            if self.under >= self.clear_n:
                self.state = IDLE
            elif self.over >= self.armed_n:
                self.state = ALARM; self.t_alarm = now; fired = True
        elif self.state == ALARM:
            if self.under >= self.clear_n and now - self.t_alarm >= self.hold:
                self.state = IDLE; fired = True
        return self.state, fired
PYEOF
```

8.3 Engine

```
cat > infer/engine.py <<'PYEOF'
import argparse, collections, queue, sys, threading, time
import numpy as np, onnxruntime as ort, sounddevice as sd, tomllib
from dsp.frontend import prep, spec, norm, residual
from infer.db import open_db, add
from infer.state import Detector

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--replay', help='wav file instead of live mic')
    a = ap.parse_args()
    cfg = tomllib.load(open('config.toml', 'rb'))

    base = np.load('models/baseline_spec.npy')
    thr = float(open(cfg['detect']['threshold_file']).read())
    sess = ort.InferenceSession('models/ae.onnx', providers=['CPUExecutionProvider'])
    det = Detector(thr, cfg['detect']['armed_frames'], cfg['detect']['clear_frames'],
                   cfg['detect']['hold_s'], cfg['detect']['clear_ratio'])
    db = open_db(cfg['log']['db'])

    q = queue.Queue(maxsize=64)
```

```
    buf = collections.deque(maxlen=cfg['dsp']['nfft'] * 4)

    def push(x):
        try:
            q.put_nowait(x)
        except queue.Full:
            q.get_nowait(); q.put_nowait(x)

    if a.replay:
        import soundfile as sf
        x, sr = sf.read(a.replay, dtype='float32')
        for i in range(0, len(x), 1024):
            push(x[i:i + 1024]); time.sleep(1024 / sr)
    else:
        def cb(d, f, t, s):
            if s: print(s, file=sys.stderr)
            push(d[:, 0].copy())
        st = sd.InputStream(device=cfg['audio']['device'] if cfg['audio']['device'] >= 0 else None,
                            channels=1, samplerate=cfg['audio']['rate'],
                            dtype='float32', blocksize=cfg['audio']['blocksize'], callback=cb)
        st.start()

    print(f'thr {thr:.6f} state idle', flush=True)
    try:
        while True:
            block = q.get()
            buf.extend(block.tolist())
            if len(buf) < cfg['dsp']['nfft'] * 2:
                continue
            x = prep(np.array(buf, dtype='float32'), cfg['audio']['rate'], cfg)
            S = norm(residual(spec(x, cfg), base))
            n = cfg['dsp']['patch']
            if S.shape[0] < n or S.shape[1] < n:
                continue
            patch = S[:n, -n:][None, None].astype('float32')
            out = sess.run(None, {'x': patch})[0]
            score = float(((out - patch) ** 2).mean())
            state, fired = det.step(score)
            print(f'{state} {score:.6f}', flush=True)
            if fired:
                add(db, state, score, thr)
                sys.stdout.write('\a' if state == 'alarm' else '')
                print(f'== {state.upper()} at {time.strftime("%H:%M:%S")} ==', flush=True)
            buf.clear()
    except KeyboardInterrupt:
        pass
    finally:
        if not a.replay:
            st.stop()

if __name__ == '__main__':
    main()
PYEOF
```

8.4 Replay test

```
python -m infer.engine --replay data/pos/$(ls data/pos | grep wav | head -1)
```

Verify: the state line reaches alarm within two seconds of a burst, and returns to idle after it stops.

8.5 Live test

```
python -m infer.engine
```

In a second terminal: `python mock/fan_morse.py --pwm "SOS"`.

> **Verify:** alarm within two seconds, clears within five, bell rings, no alarm in the quiet stretches.

8.6 Measure latency

```
python - <<'EOF'
import json, subprocess, sys, time
start = time.time()
p = subprocess.Popen([sys.executable, 'mock/fan_morse.py', '--pwm', 'SOS'])
time.sleep(1.0)
t0 = time.time()
# read the engine's stdout yourself, or use the DB rows
print('marker', t0)
p.wait()
EOF
sqlite3 events.db "select t from events where state='alarm' order by t desc limit 5"
```

Subtract the ground-truth start from the first alarm timestamp. Target median under 2000 ms.

8.7 False-alarm soak

```
timeout 600 python -m infer.engine > /tmp/soak.log
grep -c 'ALARM' /tmp/soak.log
```

> **Verify:** ten quiet minutes give 0–1 alarms. More than three means raise the threshold percentile to 99.8 and recalibrate.

8.8 Commit

```
git add infer/ && git commit -m "infer: engine, hysteresis, sqlite log, replay mode"
```

8.9 Cut

If Phase 8 overruns, drop `--replay` and the bell. Never drop the DB write.

Phase 9 — terminal UI

Target: 3 hours. **Output:** a live panel. Cut whole phase if needed; the engine prints state either way.

9.1 Waterfall widget

```
cat > ui/tui.py <<'PYEOF'
import collections, sys, time
import numpy as np
from rich.align import Align
from rich.console import Console
from rich.layout import Layout
from rich.live import Live
from rich.panel import Panel
from rich.table import Table

RAMP = ' .:-=+*#%@'

def render(hist, state, score, thr, rows=18, cols=100):
    out = []
    for r in range(rows - 1, -1, -1):
        line = ''
        for c in range(cols):
            v = hist[c][r] if c < len(hist) else 0.0
            i = min(len(RAMP) - 1, max(0, int(v * (len(RAMP) - 1))))
            line += RAMP[i]
        out.append(line)
    colour = 'red' if state == 'alarm' else ('yellow' if state == 'armed' else 'green')
    return Panel('\n'.join(out), title=f'[{colour}]{state.upper()}[/] '
                 f'score {score:.5f} thr {thr:.5f}')

def run(score_iter, thr, fps=15):
    hist = collections.deque(maxlen=100)
    state, score = 'idle', 0.0
    with Live(render(hist, state, score, thr), refresh_per_second=fps,
              screen=False) as live:
        for score, state in score_iter:
            col = np.zeros(18)
            col[:18] = np.linspace(0.2, 1.0, 18) * min(score / (thr * 2 + 1e-9), 1.0)
            hist.append(col)
            live.update(render(hist, state, score, thr))
PYEOF
```

9.2 Wire it in

```
python - <<'EOF'
p = 'infer/engine.py'
s = open(p).read()
s = s.replace("print(f'{state} {score:.6f}', flush=True)",
              "yield_pair(score, state)")
s = s.replace("from infer.state import Detector",
              "from infer.state import Detector\nfrom ui.tui import run as run_ui")
s = s.replace("    try:\n        while True:",
              "    def gen():\n        while True:")
open(p, 'w').write(s)
print('patched -> now finish the generator wiring by hand (9.3)')
EOF
```

9.3 Generator wiring

Because the engine is a blocking loop, the simplest reliable pattern is: run inference in a thread, push (score, state) pairs into a queue.Queue, and let the UI thread consume it in run_ui. Do this by hand rather than by patch — the engine already has the queue, so add a second one named pairs.

9.4 Test the panel

```
python -m infer.engine
```

Verify: the panel repaints, the title colour changes green → yellow → red, and the waterfall advances. If it tears, lower fps to 8.

9.5 Alarm banner

On alarm, swap the panel border to red and print the last five DB rows underneath. On idle, clear it.

9.6 Commit

```
git add ui/ infer/ && git commit -m "ui: waterfall panel, alarm banner"
```

Phase 10 — Rust port (cut)

Target: 2 days, hard timebox. Cut the whole phase if Phase 8 slipped. Port only if Phases 5–9 are green.

10.1 Crate

```
cargo new --bin core
cd core
cargo add cpal rustfft anyhow crossbeam-channel rusqlite ort serde \
  toml hound && cd ..
```

10.2 Config struct

```
cat > core/src/config.rs <<'EOF'
use serde::Deserialize;

#[derive(Deserialize, Clone)]
pub struct Config {
    pub audio: Audio,
    pub band: Band,
    pub detect: Detect,
}
#[derive(Deserialize, Clone)]
pub struct Audio { pub device: i32, pub rate: u32, pub blocksize: u32 }
#[derive(Deserialize, Clone)]
pub struct Band { pub lo_hz: f32, pub hi_hz: f32 }
#[derive(Deserialize, Clone)]
pub struct Detect { pub threshold_file: String, pub armed_frames: u8,
                    pub clear_frames: u8, pub hold_s: f32, pub clear_ratio: f32 }

pub fn load(p: &str) -> anyhow::Result<Config> {
    Ok(toml::from_str(&std::fs::read_to_string(p)?)?)
}
EOF
```

10.3 DSP module

```
cat > core/src/dsp.rs <<'EOF'
use rustfft::{num_complex::Complex, FftPlanner};
use std::f32::consts::PI;

pub fn hann(n: usize) -> Vec<f32> {
    (0..n).map(|i| 0.5 - 0.5 * (2.0 * PI * i as f32 / n as f32).cos()).collect()
}

pub fn mag_db(buf: &[f32], nfft: usize) -> Vec<f32> {
    let w = hann(nfft);
    let mut v: Vec<Complex<f32>> = buf.iter().zip(&w)
        .map(|(s, g)| Complex::new(s * g, 0.0)).collect();
    FftPlanner::<f32>::new().plan_fft_forward(nfft).process(&mut v);
    v[..nfft / 2].iter().map(|c| 20.0 * (c.norm() + 1e-10).log10()).collect()
}

pub fn zscore(v: &mut [f32]) {
```

```
    let m = v.iter().sum::<f32>() / v.len() as f32;
    let s = (v.iter().map(|x| (x - m).powi(2)).sum::<f32>() / v.len() as f32).sqrt() + 1e-9;
    for x in v.iter_mut() { *x = (*x - m) / s; }
}
EOF
```

10.4 Capture thread

```
cat > core/src/main.rs <<'EOF'
mod config;
mod dsp;
mod log;

use anyhow::Result;
use cpal::traits::{DeviceTrait, HostTrait, StreamTrait};
use crossbeam_channel::bounded;

fn main() -> Result<()> {
    let cfg = config::load("config.toml")?;
    let nfft = 512usize;
    let (tx, rx) = bounded::<Vec<f32>>(64);

    let host = cpal::default_host();
    let dev = host.default_input_device()
        .ok_or_else(|| anyhow::anyhow!("no input device"))?;
    let sc = cpal::StreamConfig {
        channels: 1,
        sample_rate: cpal::SampleRate(cfg.audio.rate),
        buffer_size: cpal::BufferSize::Fixed(cfg.audio.blocksize),
    };
    let stream = dev.build_input_stream(
        &sc,
        move |d: &[f32], _| { let _ = tx.try_send(d.to_vec()); },
        |e| eprintln!("audio: {e}"), None)?;
    stream.play()?;

    let db = log::open("events.db")?;
    let thr: f32 = std::fs::read_to_string(&cfg.detect.threshold_file)?.trim().parse()?;
    let mut over = 0u8;
    let mut under = 0u8;
    let mut state = "idle";

    let mut buf: Vec<f32> = Vec::with_capacity(nfft * 4);
    while let Ok(block) = rx.recv() {
        buf.extend_from_slice(&block);
        if buf.len() < nfft * 2 { continue; }

        let mut m = dsp::mag_db(&buf, nfft);
        dsp::zscore(&mut m);
        let score = m.iter().map(|x| x * x).sum::<f32>() / m.len() as f32;
        let (o, u) = (score >= thr, score < thr * cfg.detect.clear_ratio);
        over = if o { over + 1 } else { 0 };
        under = if u { under + 1 } else { 0 };

        if state == "idle" && over >= cfg.detect.armed_frames { state = "armed"; }
        if state == "armed" && under >= cfg.detect.clear_frames { state = "idle"; }
        if state == "armed" && over >= cfg.detect.armed_frames {
            state = "alarm";
            log::add(&db, state, score, thr)?;
            print!("\x07");
        }
```

```
        println!("{state} {score:.6}");
        buf.clear();
    }
    Ok(())
}
EOF
```

10.5 Build

```
cd core && cargo build --release 2>&1 | tail -20
```

Verify: compiles. Fix errors in order — do not stub the DSP.

10.6 Log module

```
cat > core/src/log.rs <<'EOF'
use rusqlite::{params, Connection};

pub fn open(p: &str) -> rusqlite::Result<Connection> {
    let c = Connection::open(p)?;
    c.execute_batch("PRAGMA journal_mode=WAL;
      CREATE TABLE IF NOT EXISTS events(
        id INTEGER PRIMARY KEY, t REAL NOT NULL, state TEXT NOT NULL,
        score REAL NOT NULL, thr REAL NOT NULL, note TEXT);")?;
    Ok(c)
}

pub fn add(c: &Connection, state: &str, score: f32, thr: f32) -> rusqlite::Result<()> {
    c.execute("INSERT INTO events(t,state,score,thr,note) VALUES(?1,?2,?3,?4,'')",
        params![std::time::SystemTime::now()
            .duration_since(std::time::UNIX_EPOCH).unwrap().as_secs_f64(),
            state, score, thr])?;
    Ok(())
}
EOF
cd core && cargo build --release
```

10.7 Parity against Python

```
python -m infer.engine --replay data/pos/test.wav > /tmp/py.txt
./core/target/release/core > /tmp/rs.txt 2>&1 &
# play the same wav into the mic, or feed it via the loopback device
diff <(awk '{print $1}' /tmp/py.txt | uniq) <(awk '{print $1}' /tmp/rs.txt | uniq)
```

Verify: the state sequences are the same shape. Exact scores will differ — the Rust version has no spectral subtraction yet, so port `residual()` before comparing numbers.

10.8 ONNX in Rust

Add ort, load `models/ae.onnx`, feed a 1×1×64×64 patch, read the output, compute MSE. Threshold comparison replaces the energy score in 10.4.

10.9 Commit

```
git add core/ && git commit -m "core: rust dsp, capture, log, state machine"
```

Phase 11 — deployment

Target: 2 hours.

11.1 Service file

```
cat > deploy/aetherhound.service <<'EOF'
[Unit]
Description=AetherHound covert channel detector
After=sound.target

[Service]
Type=simple
User=aether
WorkingDirectory=/opt/aetherhound
ExecStart=/opt/aetherhound/core
Restart=always
RestartSec=2
ProtectSystem=strict
ReadWritePaths=/opt/aetherhound
EOF
```

11.2 Install script

```
cat > deploy/install.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
test $EUID -eq 0 || { echo "run as root"; exit 1; }
id -u aether &>/dev/null || useradd -r -s /usr/sbin/nologin aether
mkdir -p /opt/aetherhound
cp -r core/target/release/core config.toml models /opt/aetherhound/
chown -R aether:aether /opt/aetherhound
cp deploy/aetherhound.service /etc/systemd/system/
systemctl daemon-reload
systemctl enable --now aetherhound
systemctl status aetherhound --no-pager
EOF
chmod +x deploy/install.sh
```

11.3 Run it

```
sudo ./deploy/install.sh
```

Verify: `systemctl status` shows active, `journalctl -u aetherhound -f` shows state lines.

11.4 Restart test

```
sudo pkill -9 core; sleep 3; systemctl is-active aetherhound
```

Verify: prints active.

11.5 Resource check

```
systemctl show aetherhound -p MainPID --value | xargs -I{} ps -o rss=,%cpu= -p {}
```

Verify: RSS under 200 MB, CPU under 25.

11.6 Commit

```
git add deploy/ && git commit -m "deploy: systemd unit and installer"
```

Phase 12 — evaluation

Target: 4 hours. Run only on `data/test.npz` and its WAVs.

12.1 Metrics

```
cat > eval/metrics.py <<'PYEOF'
import numpy as np
from sklearn.metrics import (precision_recall_fscore_support,
                             roc_auc_score, confusion_matrix)

def score_row(name, y, s, thr, hours):
    p = (s >= thr).astype(int)
    pr, rc, f1, _ = precision_recall_fscore_support(y, p, average='binary',
                                                    zero_division=0)
    tn, fp, fn, tp = confusion_matrix(y, p, labels=[0, 1]).ravel()
    ep = int(np.sum(np.diff(np.r_[0, p]) == 1))
    return dict(model=name, precision=pr, recall=rc, f1=f1,
                auc=roc_auc_score(y, s) if len(set(y)) > 1 else float('nan'),
                tp=tp, fp=fp, fn=fn, tn=tn, alarms_per_hour=ep / hours)

def table(rows):
    print(f"{'model':<12}{'prec':>7}{'rec':>7}{'f1':>7}{'auc':>7}{'alarm/h':>9}")
```

```
    for r in rows:
        print(f"{r['model']:<12}{r['precision']:>7.3f}{r['recall']:>7.3f}"
              f"{r['f1']:>7.3f}{r['auc']:>7.3f}{r['alarms_per_hour']:>9.2f}")
PYEOF
```

12.2 Run the comparison

```
cat > eval/run.py <<'PYEOF'
import numpy as np
from eval.metrics import score_row, table

d = np.load('data/test.npz'); y = d['y']
HOURS = 0.5
ae = np.load('models/test_scores.npy')
fo = np.load('models/forest_scores.npy')
thr_ae = float(open('models/threshold.txt').read())
thr_fo = float(np.percentile(fo[y == 0], 99.5))
table([score_row('autoencoder', y, ae, thr_ae, HOURS),
       score_row('isolationfst', y, fo, thr_fo, HOURS)])
PYEOF
python -m eval.run | tee docs/results.txt
```

12.3 ROC curve

```
python - <<'EOF'
import numpy as np, matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
d = np.load('data/test.npz'); y = d['y']
for name, p in (('autoencoder', 'models/test_scores.npy'),
                ('isolation forest', 'models/forest_scores.npy')):
    s = np.load(p); f, t, _ = roc_curve(y, s)
    plt.plot(f, t, label=f'{name} (AUC {auc(f, t):.3f})')
plt.plot([0, 1], [0, 1], 'k--'); plt.xlabel('FPR'); plt.ylabel('TPR')
plt.legend(); plt.tight_layout(); plt.savefig('docs/roc.png', dpi=140)
print('docs/roc.png')
EOF
```

12.4 Ablations

```
cat > eval/ablate.py <<'PYEOF'
"""Recompute test scores with one stage disabled, per run."""
import sys, numpy as np, onnxruntime as ort
from dsp.frontend import prep, spec, norm, patches, residual
import soundfile as sf, tomllib, glob

cfg = tomllib.load(open('config.toml', 'rb'))
base = np.load('models/baseline_spec.npy')
sess = ort.InferenceSession('models/ae.onnx', providers=['CPUExecutionProvider'])
mode = sys.argv[1]

def prep_variant(x, sr):
    if mode == 'no_notch':
        from dsp.filters import bandpass
        return bandpass(x, cfg['band']['lo_hz'], cfg['band']['hi_hz'], sr)
    return prep(x, sr, cfg)
```

```
scores, labels = [], []
for p in sorted(glob.glob('data/test/*.wav')):
    x, sr = sf.read(p, dtype='float32')
    S = spec(prep_variant(x, sr), cfg)
    if mode == 'no_sub':
        P = patches(norm(S), cfg['dsp']['patch'])
    else:
        P = patches(norm(residual(S, base)), cfg['dsp']['patch'])
    if not len(P):
        continue
    o = sess.run(None, {'x': P[..., None]})[0]
    scores.append(((o - P[..., None]) ** 2).mean((1, 2, 3)))
    labels.append(np.zeros(len(P)))
np.save(f'models/ablate_{mode}.npy', np.concatenate(scores))
print(mode, 'frames', sum(len(s) for s in scores))
PYEOF
python -m eval.ablate no_notch
python -m eval.ablate no_sub
```

These are negative-only runs, so read them as false-alarm rate rather than F1. Add a positive WAV directory if you want full rows.

12.5 Distance sweep

Repeat 8.6 at 1 m, 3 m and 5 m. Three rows, same format. Note the metric in the table caption.

12.6 Fill the results table

Condition	Precision	Recall	F1	AUC	Alarms/h	Latency ms
Autoencoder, 1 m						
Autoencoder, 3 m						
Autoencoder, 5 m						
Isolation forest						
Ablation, no notch						
Ablation, no subtraction						

12.7 Commit results

```
git add eval/ docs/ && git commit -m "eval: metrics, roc, ablations, results table"
```

Phase 13 — freeze and hand over

13.1 README

```
cat > README.md <<'EOF'
# AetherHound

Detects RF and acoustic covert channels in an air-gapped room by learning the
normal ambient signature and alarming on machine-like transmission.

## Run
    python -m venv .venv && source .venv/bin/activate
    pip install -r requirements.txt
    python dsp/spectral_sub.py
    python -m infer.engine

Trigger a test transmission in another shell:
    python mock/fan_morse.py --pwm "SOS"

## Train from scratch
See docs/build-manual.pdf, phases 3 to 6.

## Numbers
docs/results.txt, docs/roc.png, docs/proof.png

## Limits
Acoustic only on laptop hardware. RF path validated on synthetic I/Q.
Single room. See report section 10.
EOF
```

13.2 Tag

```
git tag -a v1.0 -m "submission" && git log --oneline | head -5
```

13.3 Record the demo video

```
ffmpeg -f x11grab -framerate 15 -i :0.0 -f alsa -i default \
  -t 90 -c:v libx264 -preset fast docs/demo.mp4
```

Then run the demo script from part 1, Section 19. Ninety seconds, no narration gaps, alarm on camera.

13.4 Archive

```
git archive --format=zip -o ../aetherhound-src.zip HEAD
du -sh data models docs
```

Ship the zip, the PDF, the video and the results file. Do not ship `data/` unless it is under 200 MB.

Compressed 5-day map

If the deadline collapses, this is the order that still produces something defensible.

Day	Phases	Ship
1	0, 1, 2	Transmitter and recorder working
2	3 (2 h only), 4	Proof plot with a visible spike
3	5, 6, 7	Model, threshold, forest row
4	8, 12	Live alarm plus results table
5	13	README, video, tag

Drop in this order: 9 (UI), 10 (Rust), 11 (deploy), 12.4 (ablations). Keep 8, 12.1 and 13.

> **Minimum shippable build:** Phases 0–6 plus 8 plus 12.1 plus 13. That is a transmitter, a recorder, a model, a live alarm, real metrics and a README. Everything else is polish.

End of manual. Phase 0, step 0.1 — start there and do not look ahead.